



北京大學
PEKING UNIVERSITY

Low-Cost and Comprehensive Non-textual Input Fuzzing with LLM-Synthesized Input Generators

基于大语言模型合成输入生成器的低成本全面非文本输入模糊测试

USENIX 2025

2026.07.28 白惠雨

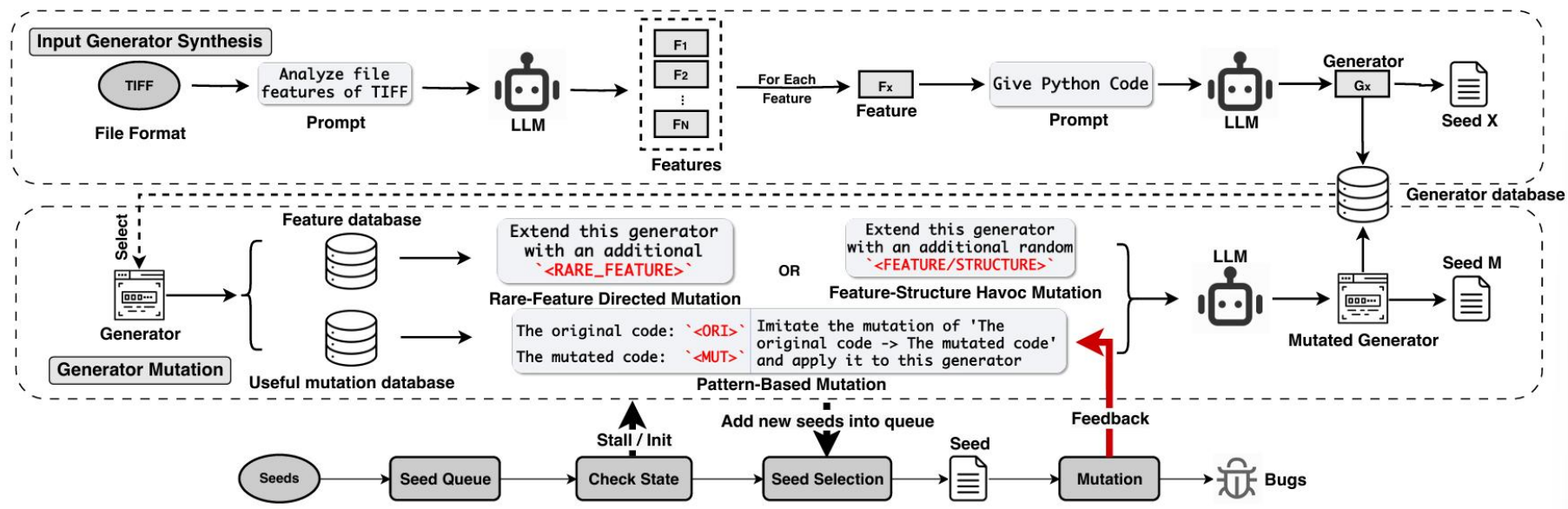
现代软件广泛接受具有高度复杂结构的**非文本输入**，如图像、音频、视频和PDF文档。为充分测试此类软件的内部逻辑，测试用例必须严格符合目标格式的语法规范，否则将在解析阶段被拒绝，无法触及深层代码。

然而，现有方法面临根本性瓶颈：**通用模糊器**（如AFL++）仅进行字节级随机变异，缺乏对格式语义的理解，生成的海量输入因结构无效而被早早拒绝，探索效率极低。**结构感知工具**（如FormatFuzzer）虽能生成有效输入，但严重依赖安全专家手工编写格式模板，面对TIFF、MP4等具有数十种复杂特征组合的格式时，扩展性差、成本高昂，且无法覆盖LZW压缩、EXIF元数据等深层特性。

近年来，**大语言模型**展现出强大的代码生成与格式理解能力，为自动化构造复杂输入提供了新可能。然而，直接让LLM生成非文本二进制文件面临生成成本高、输出多样性差、难以持续变异等现实障碍，难以独立支撑有效的模糊测试。

本文研究如何利用LLM合成输入生成器，以低成本、全自动的方式驱动非文本输入的语法感知模糊测试。

本文提出混合模糊测试框架G²FUZZ，将 LLM 驱动生成器构造机制与 AFL++ 的覆盖率引导测试循环相结合，包含生成器合成和生成器迭代变异两个核心模块。



核心流程:

- 根据目标文件格式分析文件特征，并为不同特征生成生成器；
- 利用生成器构造初始种子，交由 AFL++ 执行反馈驱动的模糊测试；
- 在探索停滞时调用 LLM 对生成器进行迭代优化，以生成更复杂的并提升后续探索能力。

模糊测试正式循环启动前执行：用户输入目标格式（如 TIFF），框架解析格式全部特征，生成对应 Python 生成器，运行脚本产出种子加入模糊队列。若软件支持多类输入格式，将逐一生成对应格式生成器。

➤ 算法1：整体流程

Algorithm 1: Generator Procedure

Input: The target file format, *target_format*.
Output: A set of (generator, feature description), *G*.

```
1 prompt  $\leftarrow$  construct_prompt(target_format)           // Based on Fig. 2
2 features  $\leftarrow$  LLM(prompt)
3 for f in features do
4   g  $\leftarrow$  generator_generation(target_format, target_feature)
                                     // Based on Alg. 2
5   if g  $\neq$  none then
6     seeds  $\leftarrow$  run(g)
7     add_to_queue(seeds)
8     G  $\leftarrow$  (g, f)
```

特征提取提示词

<TARGET>格式文件包含哪些特征？严格按照以下格式输出：

1. 特征1：特征描述
2. 特征2：特征描述
-
- N. 特征N：特征描述

➤ 算法2：生成器合成算法

Algorithm 2: Generator Synthesis Algorithm

Input: The target file format, *target_format*. The target file feature, *target_feature*.
Output: A valid generator that can generate a file with specific features, *g*.

```
1 init_cnt ← 0
2 while init_cnt < INIT_MAX do
3   dialogue ← []
4   prompt ← construct_prompt(target_format, target_feature)
                                     // Based on Fig. 3
5   dialogue.append(prompt)
6   g ← LLM(dialogue)
7   status, msg ← exec(g)
8   debug_cnt ← 0
9   while debug_cnt < DEBUG_MAX do
10    if status == SUCCESS then
11      return g
12    error_info ← get_msg(msg)
13    while TRUE do
14      if "ModuleNotFoundError" not in error_info then
15        break
16      library_prompt ← construct_prompt(error_info)
                                     // Based on Fig. 4
17      relied_library ← LLM(library_prompt)
18      flag, g = automatic_installation(relied_library)
19      if flag == 0 then
20        // Failed to install the library
21        return None
22      status, msg ← exec(g)
23      if status == SUCCESS then
24        return g
25      else
26        error_info ← get_msg(msg)
27      dialogue.append(g)
28      dialogue.append(error_info + "Regenerate")
29      G ← LLM(dialogue)
30      status, msg ← exec(g)
31      debug_cnt ← debug_cnt + 1
32    init_cnt ← init_cnt + 1
```

生成器代码提示词

使用Python生成带如下特征的<TARGET>文件，无需外部输入文件，输出保存至./tmp/目录。

目标特征：<TARGET_FEATURES>

输出仅用一个Markdown代码块，无需额外说明依赖库。

依赖库自动识别提示词

报错信息：<MSG>

请输出单行pip安装命令，仅一个Markdown代码块，用于解决该Python模块缺失问题。

核心问题处理逻辑：

- 模块缺失：LLM 解析报错返回 pip 安装命令，自动安装依赖。
- 代码语法 / 逻辑错误：将报错反馈给 LLM 迭代修复。

前一个模块生成的基础生成器仅包含单一特征，覆盖范围有限。因此本部分基于模糊测试运行反馈设计三类变异策略，生成同时包含多类特征的复杂生成器，拓展特征覆盖空间。

Algorithm 3: Generator Mutation Algorithm

Input: The target file format, *format*.
Output: A generator, *g_m*.

```
1 state ← get_fuzz_state()
2 if state == init then
3     prompt ← construct_prompt(target_format) // Based on Fig. 5
4     features ← LLM(prompt)
5     for f in features do
6         g ← generator_select()
7         prompt ← construct_prompt(format, g, f) // Based on
            Fig. 6
8         gm ← LLM(prompt)
9         gm ← self_debug(gm) // Reuse the code lines 9 - 30 in
            Alg. 2
10        seeds ← run(gm)
11        add_to_queue(seeds)
12 if state == stall then
13     g ← generator_select()
14     mutator ← mutator_choose()
15     if mutator == feature or mutator == structure then
16         prompt ← construct_prompt(format, g, mutator) // Based on
            Fig. 13
17     else if mutator == pattern then
18         example ← pre_mutation_select()
19         prompt ← construct_prompt(g, example) // Based on Fig. 7
20     gm ← LLM(prompt)
21     gm ← self_debug(gm) // Reuse the code lines 9 - 30 in
            Alg. 2
22     seeds ← run(gm)
23     add_to_queue(seeds)
```

- **稀有特征定向变异：**基于已生成特征历史，引导 LLM 生成未覆盖的稀有特征，合并至现有生成器。
- **特征 - 结构随机扰动变异：**随机向现有生成器新增任意特征 / 结构，最大化 LLM 生成潜力，产出包含大量复合特征的文件。
- **模式复用变异：**提取过往能触发新代码分支的有效变异逻辑，复用该模式改造其他生成器，聚焦对程序逻辑敏感的核心特征。

维护全局特征库，记录所有已生成的特征。每次变异时将已覆盖特征写入提示词，要求 LLM 输出遗漏的稀有特征。再随机选取一份生成器，让 LLM 在原有代码基础上新增稀有特征。该策略仅在初始化阶段执行，控制 token 开销。

稀有特征提取提示词

已解析特征列表：

1. 特征1：描述
2.

除上述特征外，<TARGET>文件还有哪些特征？严格按格式输出：

1. 特征1：描述
-

稀有特征整合提示词

下方代码用于生成<FORMAT>文件：

<TARGET_GENERATOR代码>

现在扩展代码，在原有特征基础上新增特征

<NEW_FEATURE>，该特征描述如下：

<FEATURE_DES>

不限制新增特征类型，随机向生成器添加任意特征 / 结构，可产出包含 3 种及以上复合特征的文件，弥补定向变异仅能新增单一稀有特征的局限，挖掘更多罕见格式特征。

不同特征对程序代码覆盖贡献差异极大：若某份变异后种子触发全新代码分支，则将「原生成器→变异生成器」记录为有效变异模式存入数据库。后续让 LLM 学习该修改逻辑，复用至其他生成器，批量产出高价值种子。

随机扰动提示词

基于下方代码，生成更复杂Python脚本，产出<FORMAT>文件，新增更多复杂文件特征/结构。
<TARGET_GENERATOR代码>

模式复用提示词

原始代码：<ORI>
变异后代码：<MUT>
模仿上述原始代码到变异代码的修改逻辑，复用至下方目标代码：
<TARGET_CODE>

① 实验平台：

UNIFUZZ：开源、指标标准化的模糊测试综合评测平台

② 被测程序：

仅接收非文本输入的 10 个程序，覆盖 20 多种输入格式

③ 对比基线（4 类主流 AFL++ 配置）：

- AFL++(cmplog)：启用 Redqueen 变异模块
- AFL++(mopt)：启用 MOPT 变异调度
- AFL++(fast)：启用 AFLFast 种子调度
- AFL++(rare)：优先稀有覆盖种子

④ 评估指标：

- 分支覆盖率
- 唯一崩溃数量

UNIFUZZ

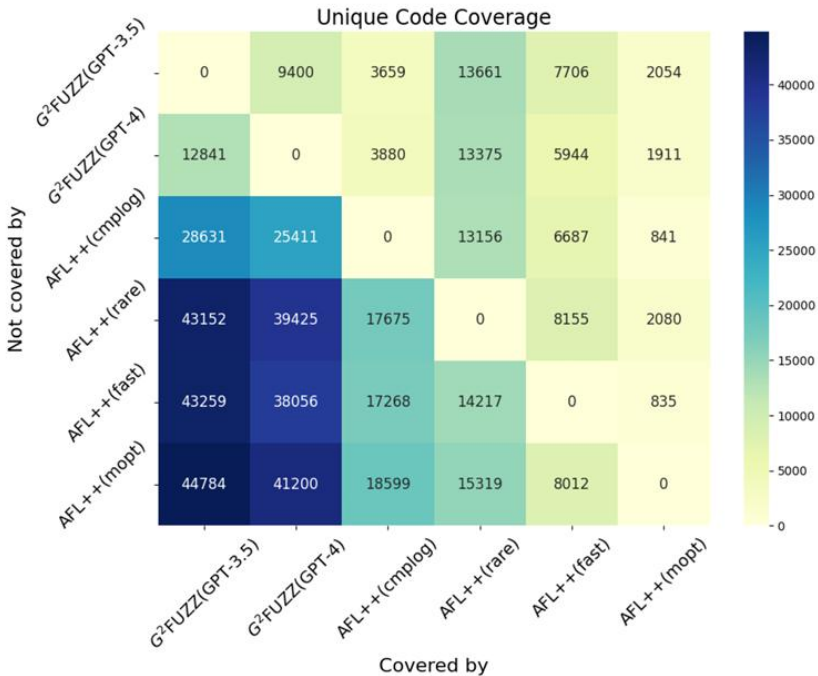
gdk-pixbuf-pixdata
jhead
mp3gain
ffmpeg
tiffsplit
pdftotext
mp42aac
flvmeta
imginfo
exiv2

RQ1: 对比实验-平台1

Programs	G ² Fuzz(GPT-3.5) ¹		G ² Fuzz(GPT-4) ²		AFL++(cmplog)		AFL++(fast)		AFL++(mopt)		AFL++(rare)	
	Cov.	#Bug	Cov.	#Bug	Cov.	#Bug	Cov.	#Bug	Cov.	#Bug	Cov.	#Bug
exiv2	5,099	31	5,171	28	4,965	26	3,776	5	3,758	12	3,851	11
ffmpeg	31,706	0	34,218	1	22,099	0	17,380	0	15,566	0	14,613	0
flvmeta	228	4	228	4	228	4	228	4	228	4	228	4
gdk	2,958	6	2,327	2	2,172	5	2,093	4	2,082	2	1,991	4
imginfo	2,839	0	3,825	0	2,189	0	1,998	0	2,004	0	1,976	0
jhead	315	13	491	23	445	21	195	3	195	3	195	4
mp3gain	921	11	921	12	923	11	900	10	899	8	891	10
mp4aac	2,067	17	2,700	14	2,091	13	1,178	6	1,157	3	1,135	2
pdftotext	8,265	43	7,921	49	7,434	25	6,483	25	6,376	28	11,257	28
tiffsplit	1,817	7	1,840	10	1,659	6	1,619	9	1,644	9	1,626	7

¹ G²Fuzz(GPT-3.5): G²Fuzz based on GPT-3.5.

² G²Fuzz(GPT-4): G²Fuzz based on GPT-4.



• 分支覆盖率：G²Fuzz (GPT-4) 和 G²Fuzz (GPT-3.5) 在10个程序中的9个上取得了最高性能，AFL++(cmplog) 在一个程序上表现最好。

• 漏洞数量：G²Fuzz (GPT-4) 发现了 143 个漏洞，比表现最好的对比基线 AFL++(cmplog) 多32 个。

• 独特代码覆盖率：G²Fuzz (GPT-4) 和G²Fuzz (GPT-3.5) 都能识别比其余四个对比基线更多的独特代码覆盖率，说明G²Fuzz生成的具有复杂特性的输入能够触发更复杂的程序逻辑。

① 实验平台：

FuzzBench：由 Google 开发，每套“模糊器 - 被测程序”组合独立 Docker 容器运行

② 被测程序：

仅接收非文本输入的 11 个程序

③ 对比基线：

AFL++、LibAFL、LibFuzzer、HonggFuzz、
AFLSmart、AFL、MOPT、Eclipser、FairFuzz、
AFLFast、Centipede 共 11 个

④ 评估指标：

- 中位数代码覆盖率：排名数值越低性能越好，取多程序平均排名作为综合指标

FuzzBench

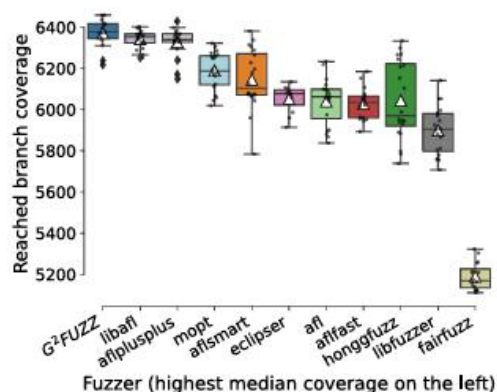
bloaty_fuzz_target
freetype2-2017
harfbuzz-1.3.2
lcms-2017-03-21
libjpeg-turbo-07-2017
libpcap_fuzz_both
libpng-1.2.56
openssl_x509
vorbis-2017-12-11
woff2-2016-05-06
zlib_zlib_uncompress_fuzzer

RQ1: 对比实验-平台2

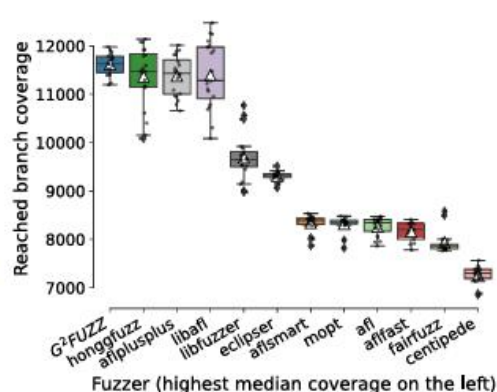
为了减轻 LLM 生成的随机性，每组实验重复 3 轮，生成 3 套独立种子

Fuzzers	Group I	Group II	Group III	Average
G ² FUZZ	2.09	2.18	2.18	2.15
AFL++	2.73	2.73	2.91	2.79
LibAFL	4.55	4.55	4.64	4.58
LibFuzzer	4.73	4.64	4.64	4.67
Honggfuzz	6.45	6.45	6.36	6.42
AFLSmart	6.73	6.73	6.73	6.73
AFL	7.27	7.27	7.27	7.27
MOPT	7.27	7.27	7.27	7.27
Eclipser	7.36	7.36	7.27	7.3
FairFuzz	8.82	8.82	8.82	8.82
AFLFast	9.09	9.09	9.09	9.09
Centipede	9.18	9.18	9.18	9.18

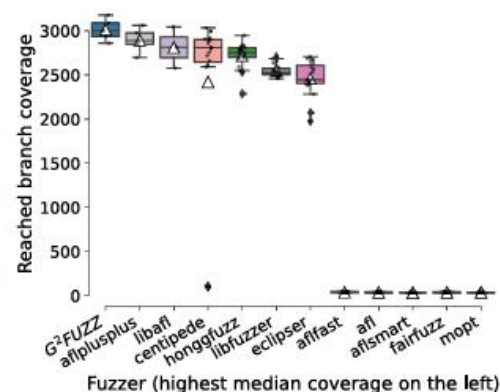
- 平均排名：G²FUZZ 在所有三组中都取得了中位数代码覆盖率的最佳排名，分别为 2.09、2.18 和 2.18
- 分支覆盖率分布：11 个被测程序中，G²FUZZ 在 5 个程序上取得最高覆盖率，其他工具只有1/2个。说明 G²FUZZ 能够发现传统模糊器难以覆盖的代码区域。



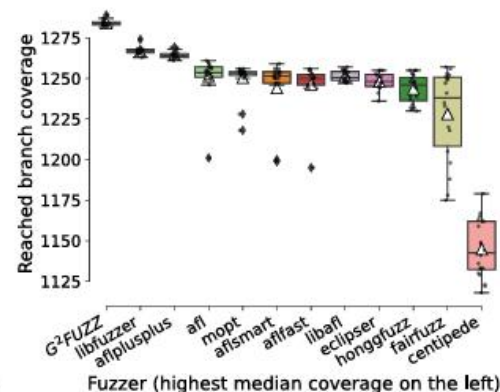
(a) bloaty



(b) freetype2



(c) libpcap



(d) vorbis

① 实验平台：

MAGMA：带有真实世界漏洞的模糊测试基准

② 被测程序：

仅接收非文本输入的 6 个程序

③ 对比基线：

AFL++、MOPT、AFLFast、LibFuzzer、Entropic共 5 个
每组实验重复5次

④ 评估指标：

- 漏洞数量

MAGMA

libpng_read_fuzzer
read_rgba_fuzzer
tiffcp
pdf_fuzzer
pdfimages
pdftoppm
sndfile_fuzzer

Table 3: The total CVEs discovered (on MAGMA). G²FUZZ performs the best in discovering real CVEs.

Program	G ² FUZZ	AFL++	MOPT	AFLFast	LibFuzzer	Entropic
libpng_read_fuzzer	3	3	2	1	3	0
tiff_read_rgba_fuzzer	5	5	5	4	2	3
tiffcp	7	6	4	4	0	0
pdf_fuzzer	5	5	4	2	2	2
pdftoppm	6	5	6	2	0	0
pdfimages	6	5	5	3	0	0

- 漏洞数量: G²Fuzz 表现最好, 在所有程序中发现了最多的漏洞。分别在libpng_read_fuzzer、tiff_read_rgba_fuzzer、tiffcp、pdf_fuzzer、pdftoppm 和 pdfimages上发现了3、5、7、5、6和6个漏洞。

RQ2: 性能是否超越各类结构感知模糊器

使用 UNIFUZZ 基准将 G²Fuzz 与语法感知模糊器 FormatFuzzer 和基于推断的模糊器 WEIZZ 进行比较

Table 6: The average line coverage discovered by G²FUZZ, FormatFuzzer, and WEIZZ.

Programs	G ² FUZZ		FormatFuzzer		WEIZZ	
	line	function	line	function	line	function
exiv2	5,984	1488	1,138	369	3,732	1025
ffmpeg	53,664	3028	23,114	1554	26,789	1795
flvmeta	623	59	- ¹	-	632	60
imginfo	5,003	364	2,128	193	3,481	275
jhead	431	21	239	16	300	18
mp3gain	2,168	58	# ²	#	2,103	56
mp4aac	3,378	811	#	#	2,041	504
pdftotext	13,733	1182	-	-	9,133	914
tiffsplit	3,176	194	-	-	3,019	185
gdk	4,856	315	2,287	192	= ³	=

¹ -: FormatFuzzer does not support PDF, TIFF, and FLV formats.

² #: We encountered issues while running FormatFuzz.

³ =: We are unable to compile *gdk-pixbuf* by WEIZZ.

- 行覆盖率：在10个程序中的9个中，G²Fuzz 实现了更高的行覆盖率，且可以扩展到更广泛接受不同格式的程序，利于生成输入的多样性。
- 独有发现的函数数量：在10个程序中的9个中，G²Fuzz 识别出了最多的独有函数，说明其触发更细微程序逻辑的能力更强，证明了使用 LLMs 生成复杂二进制输入的有效性。

RQ3: 连续 24 小时模糊测试会消耗多少 token、产生多少成本

和RQ1中UNIFUZZ平台中的实验设置相同

Table 12: Token consumption and cost analysis for 24 hours of fuzzing in UNIFUZZ.

Programs	G ² FUZZ(GPT-3.5)		G ² FUZZ(GPT-4)	
	Token Count	Cost(\$)	Token Count	Cost(\$)
ffmpeg	57,870.0	0.10	112,763.8	3.97
gdk	80,720.4	0.14	186,810.8	6.50
jhead	68,816.2	0.12	158,385.6	5.68
mp42aac	64,593.6	0.12	170,129.4	6.03
tiffsplit	75,616.6	0.14	156,247.8	5.58
exiv2	41,958.4	0.08	100,312.0	3.58
flvmeta	52,277.4	0.09	350,607.6	12.71
imginfo	66,237.4	0.12	138,265.4	4.78
mp3gain	108,270.2	0.20	152,863.2	5.50
pdftotext	69,787.6	0.13	131,047.2	4.64

- **GPT-3.5:** 单程序 24 小时最高花费 0.2 美元，全部程序均低于 0.2 美元
- **GPT-4:** 单程序最高花费 12.71 美元，全部程序均低于 13 美元
- 整体调用成本可控

RQ4: G²Fuzz两大核心模块, 哪一部分对覆盖提升贡献更大

和RQ1中UNIFUZZ平台中的实验设置相同, 统计通过对每个组件的种子进行变异而发现的新路径数量。

Table 13: The number of new pathes contributed by the different components of G²FUZZ.

Programs	Initial Seeds	Input Generator Synthesis	Generator Mutation
tiffsplit	101	539	2,549
jhead	2	1,046	0
mp42aac	6,859	522	6,298
gdk	9,832	12,993	1
ffmpeg	4,877	14,603	109,381
exiv2	398	40,719	18,328
flvmeta	1,133	1,066	29
imginfo	21,236	593	0
mp3gain	1,675	1,377	4,266
pdftotext	7,735	8,543	488
Total	53,848	82,001	141,340

- 输入生成器合成: 总计新增 82001 条路径
- 生成器变异: 总计新增 141340 条路径, 多数程序中生成器变异贡献更高

➤ 文件特征覆盖实验

Table 10: The feature coverage covered by each fuzzer.

Program	G ² Fuzz	AFL++(cmplog)	AFL++(mopt)	AFL++(fast)	AFL++(rare)
TIFF (tiffsplit)	4719	2303	2169	2369	2178
JPG (exiv2)	4096	1910	1910	1913	1909
MP4 (mp42aac)	1459	0	0	0	0
PDF (pdftotext)	37797	33451	40730	44704	36203

- G²Fuzz (GPT-4) 在 TIFF、JPG 和 MP4 上取得了最高的特征覆盖。AFL++ 更侧重于底层变异，难以修改高层特征。更改高层特征需要同时处理多个数据块之间的约束，这对字节级变异来说具有很强的挑战性。

➤ 多LLM模型通用性验证

Table 11: Functions exclusively discovered by each fuzzer.

Format	GPT-3.5	GPT-4	llama-3-8b-instruct	llama-3-70b-instruct
JPG	259	984	211	636
MP4	- ¹	290	245	517
PDF	374	559	555	504
TIFF	388	387	591	516

¹ -: GPT-3.5 cannot generate valid MP4 files during the first round due to randomness.

- GPT-4 在 JPG 和 PDF 上取得了最高的特征覆盖，开源模型 llama-3-8b-instruct 和 llama-3-70b-instruct 分别在 TIFF 和 MP4 上取得了最高的特征覆盖。表明 G²Fuzz 的可扩展性及其使用开源模型生成高质量生成器的能力。



感谢各位老师同学批评与指导

THANKS TO THE TEACHER'S CAREFUL GUIDANCE